# Teste de integração

## Do teste unitário e TDD à colaboração entre objetos



**Prof. Dr. João Choma**
Universidade Estadual de Maringá - DIN/CTC
github.com/JoaoChoma

Teste e Qualidade de Software ▪ Semana 12 / 2026



## O próximo passo depois do teste unitário e do TDD

Uma unidade pode funcionar sozinha. Precisamos verificar também se os componentes colaboram corretamente.

- Identificar a fronteira entre teste unitário e integração.
- Testar duas classes reais com Java e JUnit 5.
- Verificar o retorno da operação e seu efeito no estado.
- Executar os cenários com e sem estoque suficiente.

## Cenário: um pedido depende do estoque

Um pedido só pode ser realizado quando houver estoque suficiente.




- **PedidoService**: coordena a realização do pedido.
- **EstoqueService**: consulta e atualiza a quantidade disponível.

Sem banco de dados, Spring ou mocks neste primeiro exemplo.

## Contrato dos cenários que vamos testar



| Estoque inicial | Pedido | Retorno | Estoque final |
|---|---|---|---|
| 10 unidades | 3 unidades | `true` | 7 unidades |
| 2 unidades | 3 unidades | `false` | 2 unidades |

**Duas observações em cada teste**

O pedido foi aceito ou recusado? O estoque ficou no estado esperado?

Neste recorte, usamos quantidades positivas. Validação de entradas fica como extensão da prática.

# EstoqueService: estado e operações



```java
package br.edu.exemplo;
public class EstoqueService {
    private int quantidade;
    public EstoqueService(int quantidadeInicial) {
        this.quantidade = quantidadeInicial;
    }
    public boolean temEstoque(int quantidadeSolicitada) {
        return quantidade >= quantidadeSolicitada;
    }
    public void retirar(int quantidade) {
        this.quantidade -= quantidade;
    }
    public int getQuantidade() {
        return quantidade;
    }
}
```

Versão corrigida: `this.quantidade` identifica o atributo do objeto.

# Um defeito no anexo: o teste deve revelá-lo



```java
// Trecho original do anexo
public void retirar(int quantidade) {
    quantidade -= quantidade;
}
```

- O parâmetro tem o mesmo nome do atributo e o oculta.
- A subtração altera apenas o parâmetro local; o estoque não muda.
- Com estoque 10 e retirada 3, o resultado observado é 10, não 7.

```java
// Correcao: atualizar o estado do objeto
this.quantidade -= quantidade;
```

**Conexão com TDD:** teste falhando, correção mínima, testes verdes e revisão do código.

# PedidoService: coordenando a regra



```java
package br.edu.exemplo;
public class PedidoService {
    private EstoqueService estoqueService;
    public PedidoService(EstoqueService estoqueService) {
        this.estoqueService = estoqueService;
    }
    public boolean realizarPedido(int quantidade) {
        if (!estoqueService.temEstoque(quantidade)) {
            return false;
        }
        estoqueService.retirar(quantidade);
        return true;
    }
}
```

O construtor recebe o estoque que será consultado e atualizado.

## Como os objetos colaboram



1. O teste chama `pedido.realizarPedido(3)`.
2. O pedido consulta `estoque.temEstoque(3)`.
3. Se houver saldo, chama `estoque.retirar(3)`.
4. O estoque passa de 10 para 7 e o pedido retorna `true`.

### O caminho de recusa

Se a consulta retornar `false`, o pedido retorna `false` imediatamente. Não deve ocorrer retirada.

O teste observa a colaboração pelo retorno e pelo estado final do estoque.

# Primeiro: teste unitário do estoque



```java
package br.edu.exemplo;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class EstoqueServiceTest {
    @Test
    void deveInformarQuandoExisteEstoque() {
        EstoqueService estoque = new EstoqueService(10);
        boolean resultado = estoque.temEstoque(3);
        assertTrue(resultado);
    }
    @Test
    void deveRetirarQuantidadeDoEstoque() {
        EstoqueService estoque = new EstoqueService(10);
        estoque.retirar(3);
        assertEquals(7, estoque.getQuantidade());
    }
}
```

Uma unidade sob teste: `EstoqueService`. Não há pedido neste teste.

## Lendo o teste: preparar, agir e verificar



| | |
|---|---|
| Preparar | Criar um estoque com 10 unidades. |
| Agir | Consultar a disponibilidade ou retirar 3 unidades. |
| Verificar | Comparar o resultado observado com o esperado. |

**Por que o segundo teste é essencial?**

Consultar a disponibilidade não prova que a retirada funciona. O defeito do anexo passa no primeiro teste e falha no segundo.

Cada método cria seu próprio objeto: um teste não depende do estado deixado pelo outro.

# Integração: pedido aceito e estoque atualizado



```java
package br.edu.exemplo;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class PedidoServiceIntegrationTest {
    @Test
    void deveRealizarPedidoERetirarProdutoDoEstoque() {
        // Componente real
        EstoqueService estoque = new EstoqueService(10);
        // Componente real utilizando o estoque
        PedidoService pedido = new PedidoService(estoque);
        // Executa a operacao
        boolean realizado = pedido.realizarPedido(3);
        // Verifica o pedido e o efeito no outro componente
        assertTrue(realizado);
        assertEquals(7, estoque.getQuantidade());
    }
}
```

## O que torna este teste de integração?



- Instanciamos **dois componentes reais**.
- Entregamos o estoque ao pedido pelo construtor.
- Executamos a operação pela interface de `PedidoService`.
- Verificamos seu retorno e o efeito em `EstoqueService`.

**Pedido realizado?**

`true`

**Estoque atualizado?**

$10 - 3 = 7$

A classificação considera a fronteira definida para o teste; aqui, testamos a integração entre essas duas classes.

## Integração: recusa sem alterar o estoque



Adicionar à classe `PedidoServiceIntegrationTest`:

```
@Test
void naoDeveRealizarPedidoSemEstoqueSuficiente() {
    EstoqueService estoque = new EstoqueService(2);
    PedidoService pedido = new PedidoService(estoque);

    boolean realizado = pedido.realizarPedido(3);

    assertFalse(realizado);
    // O estoque deve permanecer inalterado
    assertEquals(2, estoque.getQuantidade());
}
```

### Recusar também tem um efeito esperado

Retornar `false` é necessário, mas não basta: o saldo deve continuar em 2.

# Unitário e integração respondem a perguntas distintas



| Teste | O que verificamos? | Neste exemplo |
|---|---|---|
| Unitário | A unidade funciona isoladamente? | Consulta e retirada em EstoqueService. |
| Integração | Componentes reais colaboram corretamente? | PedidoService utiliza EstoqueService e preserva a regra. |

**Ideia central**

Teste de integração não exige necessariamente banco de dados ou API. O foco é a colaboração entre componentes reais.

# Estrutura Maven da prática



```
teste-integracao/
|-- pom.xml
`-- src/
    |-- main/java/br/edu/exemplo/
    |   |-- EstoqueService.java
    |   `-- PedidoService.java
    `-- test/java/br/edu/exemplo/
        |-- EstoqueServiceTest.java
        `-- PedidoServiceIntegrationTest.java
```

- Classes de produção em `src/main/java`.
- Classes de teste em `src/test/java`.
- Mesmo pacote: `br.edu.exemplo`.

## JUnit 5 e execução dos testes



No `pom.xml`, dentro de `dependencies`:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.4</version>
    <scope>test</scope>
</dependency>
```

Na pasta que contém o `pom.xml`:

```
mvn test
```

- Reutilize o projeto Maven/JUnit das práticas anteriores.
- Mantenha o Surefire compatível com JUnit 5.
- Ambas as classes terminam em `Test`: a execução deve incluir os quatro testes apresentados.

## Prática guiada: da falha à integração



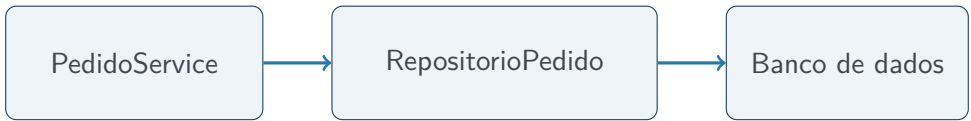
1. Implemente o estoque com o trecho original de `retirar`.
2. Execute os dois testes unitários e explique a falha observada.
3. Corrija a atualização do atributo e execute novamente.
4. Adicione o pedido e os dois testes de integração.
5. Confira retorno e saldo nos cenários de aceitação e recusa.

### Desafio

Com estoque inicial 3 e pedido de 3, qual deve ser o retorno? Qual deve ser o saldo? Escreva o teste antes de executá-lo.

Resultado esperado do desafio: `true` e estoque final 0.

## Próximo passo: integração com persistência




- Ampliar a fronteira: serviço, repositório e banco.
- H2: uma opção de banco em memória para a prática.
- Testcontainers: executar dependências em contêineres nos testes.
- Preparar dados e limpar o estado entre execuções.

**O princípio permanece**

Verificar se os componentes reais colaboram e produzem o efeito esperado.

## Fechamento: o que o teste precisa demonstrar?



- **Unidade**: o estoque consulta e atualiza seu próprio estado.
- **Integração**: o pedido usa o estoque real corretamente.
- **Aceitação**: retorno verdadeiro e saldo reduzido.
- **Recusa**: retorno falso e saldo preservado.
- **TDD**: observar a falha, corrigir e manter os testes verdes.

### Pergunta para discussão

Se o pedido retorna `true`, mas o estoque continua em 10, o cenário passou? Qual asserção evidencia o defeito?

Material-base: anexo fornecido para esta aula. Correção do sombreamento do atributo explicitada; prática guiada acrescentada para discussão.